

Part 1: Database Schema (SQL/Relational)

Here is the Entity-Relationship diagram in text format. I recommend using **UUIDs** for IDs to avoid enumeration attacks and merge conflicts.

Core Tables

sql

-- 1. Users (All actors)

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  full_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(50),  
  role ENUM('customer', 'support_agent', 'consultant', 'admin'),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- 2. Customers (extends users)

```
CREATE TABLE customers (  
  id UUID PRIMARY KEY REFERENCES users(id),  
  company_name VARCHAR(255),  
  address TEXT,  
  preferred_contact_method ENUM('email', 'phone', 'whatsapp')  
);
```

-- 3. Incidents (Tickets)

```
CREATE TABLE incidents (  
  id UUID PRIMARY KEY,  
  ticket_number SERIAL UNIQUE, -- Human readable ID  
  customer_id UUID REFERENCES customers(id),  
  created_by UUID REFERENCES users(id), -- support agent  
  
  -- Problem details  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  priority ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',  
  status ENUM('open', 'assigned', 'resolved_remote', 'pending_intervention', 'closed') DEFAULT 'open',
```

```

-- Assignment
assigned_to UUID REFERENCES users(id), -- consultant
assigned_at TIMESTAMP,

-- Resolution tracking
resolution_notes TEXT, -- If resolved away
resolved_at TIMESTAMP,

created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);

-- 4. Interventions (On-site visit)
CREATE TABLE interventions (
    id UUID PRIMARY KEY,
    incident_id UUID REFERENCES incidents(id) ON DELETE CASCADE,

    -- Scheduling
    scheduled_date DATE,
    start_time TIME,
    end_time TIME,

    -- Work details
    work_description TEXT NOT NULL, -- Rapport of works done
    materials_used TEXT, -- JSON or comma separated

    -- Status
    status ENUM('scheduled', 'in_progress', 'completed', 'cancelled') DEFAULT 'scheduled',

    created_at TIMESTAMP DEFAULT NOW()
);

-- 5. Signatures (Separate table for audit trail)
CREATE TABLE signatures (
    id UUID PRIMARY KEY,
    intervention_id UUID REFERENCES interventions(id) ON DELETE CASCADE,
    signer_type ENUM('customer', 'consultant'),
    signer_id UUID REFERENCES users(id),
    signature_data TEXT NOT NULL, -- Base64 of canvas, or path to image
    signed_at TIMESTAMP DEFAULT NOW(),
    ip_address INET

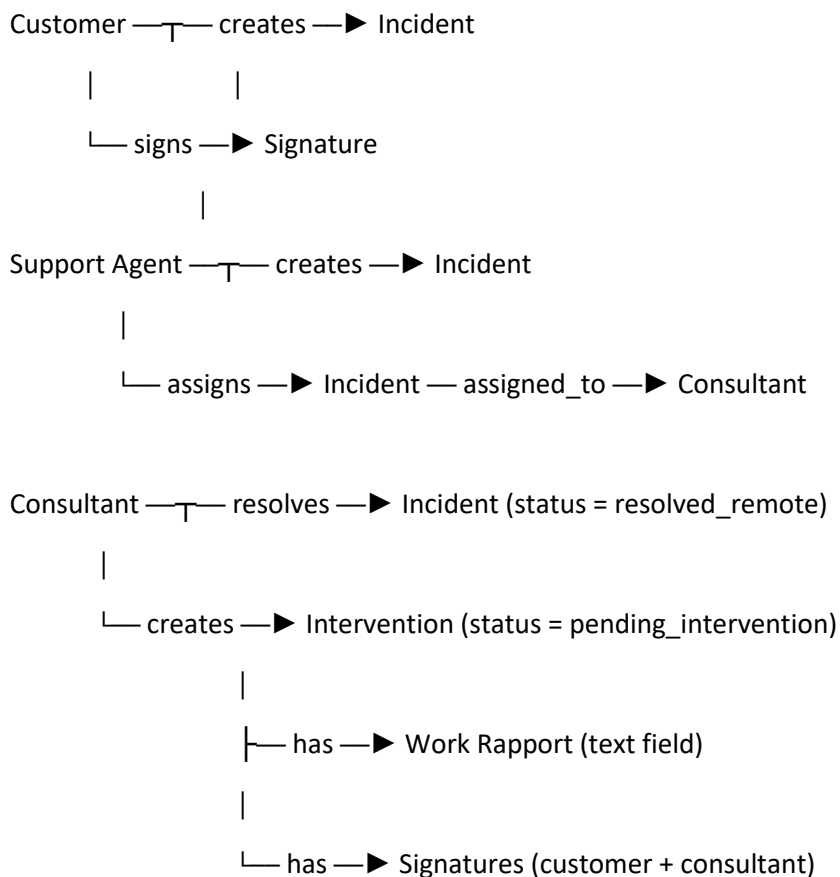
```

```
);

-- 6. Audit Log (Optional but recommended)
CREATE TABLE incident_history (
    id UUID PRIMARY KEY,
    incident_id UUID REFERENCES incidents(id),
    changed_by UUID REFERENCES users(id),
    old_status VARCHAR(50),
    new_status VARCHAR(50),
    comment TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

//////////
```

Relationships Diagram



Step	Actor	Action	System State Change
1	Customer	Calls or creates ticket	<code>Incident.status = open</code>
2	Support Agent	Creates Incident & assigns to Consultant	<code>Incident.status = assigned</code> <code>Incident.assigned_to = consultant_id</code>
3	Consultant	Receives notification, contacts customer	No change
4	Consultant	Decision <i>If solved remotely</i> → Add notes <i>If needs on-site</i> → Create Intervention	<code>Incident.status = resolved_remote</code> <code>Incident.resolution_notes = "..."</code> <code>Incident.status = pending_intervention</code> Create <code>Intervention</code> record
5	Consultant	Performs intervention, fills work rapport	<code>Intervention.work_description = "..."</code>
6	Customer + Consultant	Sign digitally	Create <code>Signature</code> records (2 rows)
7	Consultant	Closes intervention	<code>Intervention.status = completed</code> <code>Incident.status = closed</code>

2. Required API Endpoints (REST or GraphQL)

Method	Endpoint	Description
POST	<code>/api/incidents</code>	Customer or Support creates a ticket
PUT	<code>/api/incidents/:id/assign</code>	Support assigns consultant
PATCH	<code>/api/incidents/:id/resolve-remote</code>	Consultant resolves away
POST	<code>/api/incidents/:id/intervention</code>	Consultant creates an intervention
PUT	<code>/api/interventions/:id/rapport</code>	Consultant adds work description
POST	<code>/api/interventions/:id/signature</code>	Add signature (customer or consultant)
GET	<code>/api/incidents/:id/timeline</code>	View full history + signatures

3. Frontend Pages (MVP)

- **Dashboard** (Role-based view)
 - Customer: My open tickets
 - Support Agent: Unassigned tickets
 - Consultant: My assigned tickets + pending interventions
- **Ticket Detail Page**
 - Conversation/notes
 - Button "Resolve Remotely" (for consultant)
 - Button "Create Intervention" (for consultant)
- **Intervention Form**
 - Date picker
 - Rich text editor for "Rapport of works done"
 - Signature pad (HTML Canvas)
 - Customer signature field + Consultant signature field